

Senior Backend Engineer

Must-Practice Scenario Interview Questions

Focused, question-only preparation set for Python / FastAPI / TypeScript / Node.js / NestJS / PostgreSQL / MongoDB / Redis / Kafka / Nginx / AWS / Docker / production engineering.

Selection principle: high-signal questions that repeatedly exercise senior-level production reasoning: debugging, async execution, dependency failure, API design, concurrency, database behavior, distributed systems, cloud architecture, security, and technical ownership.

Total: 120 questions across 12 core domains.

1. Production Debugging & Incident Investigation

1. Your request flow is DNS -> AWS Load Balancer -> Nginx -> EC2 -> Docker -> application. You receive a 502 response. How will you debug it step by step?
2. Users intermittently receive 504 responses, while application health checks remain green. How will you determine where the timeout occurs?
3. Nginx is running, but one API route returns the Nginx default 404 page instead of the backend response. How will you trace the request?
4. An application works when called directly on its port but fails through the load balancer. What would you check and in what order?
5. One EC2 instance behind the load balancer is failing while the others work normally. How would you isolate the fault?
6. Production API latency increases from 100 ms to 2 seconds immediately after a deployment. How would you prove whether the deployment caused it?
7. Your API returns 200 responses but users report that requests occasionally hang until the client times out. How would you investigate?
8. The service has normal CPU and memory but p95 and p99 latency suddenly increase. What signals would you inspect?
9. A container repeatedly restarts in production. How would you determine whether the cause is OOM, health checks, process crashes, or infrastructure?
10. A production problem occurs only during high concurrency and cannot be reproduced with one request. How would you investigate it?

2. Async Python / FastAPI / Node.js / TypeScript

1. A class has an API method wrapped by a logging function, but the logging code runs immediately instead of after the API response. What could be wrong?
2. A function calls an async method without await. What behavior would you expect?
3. A decorator logs execution time as almost zero even though the wrapped API takes several seconds. How would you debug the decorator?
4. A logging wrapper prints 'completed' before an async method actually completes. How would you redesign it?
5. A try/catch catches synchronous errors but misses failures from an async operation. What is wrong?
6. A Node.js service uses `array.forEach(async item => ...)` and expects the caller to wait for all operations. What is wrong?
7. You need to call five independent third-party APIs from a FastAPI endpoint. How would you execute them?
8. You need to process 10,000 async operations but only want 20 in flight at any time. How would you design the concurrency limit?
9. A Node.js endpoint uses `Promise.all` for 1,000 outbound requests and production becomes unstable. What could be happening?
10. An async endpoint performs CPU-heavy work and blocks unrelated requests. How would you redesign it?

3. Third-Party APIs & Dependency Failures

1. Your application integrates three third-party APIs and one becomes slow. How would you prevent it from degrading the entire request path?
2. A third-party API normally responds in 200 ms but sometimes takes 15 seconds. How would you design the integration?

3. A payment provider times out, but you do not know whether the payment was actually processed. How would you design the workflow?
4. An external API returns intermittent 5xx errors. How would you decide whether and how to retry?
5. When can retries make an outage worse?
6. One dependency is unavailable for 30 minutes. How should your application behave for requests that depend on it?
7. How would you implement circuit breaking for an unreliable dependency?
8. Two external providers offer the same capability. How would you design failover between them?
9. A provider imposes strict rate limits and your traffic suddenly increases 20x. How would you protect your system?
10. How would you make an external API integration idempotent from your application's point of view?

4. API Design & Backend Architecture

1. How would you design an idempotent create-order API?
2. An API operation can take 20 minutes. Would you keep the HTTP request open? How would you design it?
3. How would you design an asynchronous job-submission API with job status tracking?
4. How would you design pagination for a table containing hundreds of millions of rows?
5. When would you choose cursor pagination over offset pagination?
6. How would you design consistent error responses across a large FastAPI/NestJS codebase?
7. How would you separate validation errors, business errors, dependency errors, and unexpected errors?
8. One endpoint is generating excessive database traffic. How would you redesign it?
9. How would you structure a large FastAPI or NestJS application so controllers do not contain business logic?
10. How would you decide what belongs in the synchronous request path versus asynchronous processing?

5. PostgreSQL: Performance, Transactions & Concurrency

1. A PostgreSQL query is fast with one million rows but becomes slow with one hundred million. How would you investigate?
2. How would you use EXPLAIN or EXPLAIN ANALYZE to debug a slow PostgreSQL query?
3. Two users try to purchase the last available item at exactly the same time. How would you prevent overselling?
4. When would you use optimistic locking versus pessimistic locking?
5. A read-then-update workflow produces lost updates under concurrency. How would you redesign it?
6. How would you choose between READ COMMITTED, REPEATABLE READ, and SERIALIZABLE for a business operation?
7. Two transactions deadlock in production. How would you investigate and reduce the likelihood of future deadlocks?
8. Your application opens far more PostgreSQL connections than expected. How would you find the source and fix it?
9. How would you perform a schema migration or index creation on a large production table without unacceptable downtime?
10. How would you design a zero-downtime database migration when old and new application versions must run simultaneously?

6. MongoDB: Modeling, Indexing & Scaling

1. A MongoDB collection has 100 million documents and a frequently used query became slow. What would you inspect first?
2. How would you decide between embedding and referencing for a MongoDB document model?
3. An aggregation pipeline is slow in production. How would you identify the expensive stage and optimize it?
4. How would you choose MongoDB indexes from real query patterns rather than from the schema alone?
5. A document contains an array that grows indefinitely and is frequently queried. How would you redesign the model?
6. How would you choose a shard key for a high-write MongoDB workload?
7. What characteristics can make a MongoDB shard key a hotspot?
8. A query becomes scatter/gather in a sharded cluster. What does that mean for performance and how would you reduce it?
9. You need to update tens of millions of MongoDB documents. How would you make the migration safe, resumable, and observable?
10. How would you design multi-tenant data isolation in MongoDB?

7. Redis, Caching & Rate Limiting

1. When should an API use Redis instead of querying PostgreSQL directly?
2. Redis goes down during peak traffic. How should the application degrade?
3. How would you implement cache-aside for a read-heavy endpoint?
4. How would you handle stale cache data after a PostgreSQL update?
5. What is a cache stampede scenario and how would you prevent it?
6. A popular Redis key becomes a hot key. How would you handle it?
7. How would you choose TTLs for data with different freshness requirements?
8. How would you design a distributed per-user and per-IP rate limiter using Redis?

9. Two application instances update the same Redis-based counter concurrently. How would you make the operation safe?
10. How would you prevent a cache outage from causing a database avalanche?

8. Kafka / Messaging / Event-Driven Architecture

1. A Kafka consumer is processing messages slower than producers are publishing them. How would you diagnose and scale it?
2. A consumer processes the same event twice. How should the application handle duplicates?
3. A consumer updates PostgreSQL and crashes before committing its Kafka offset. What could happen?
4. How would you design an idempotent Kafka consumer?
5. How would you choose a Kafka partition key for an order or user domain?
6. One Kafka partition is receiving much more traffic than the others. How would you diagnose and fix the imbalance?
7. How would you design retries without allowing one poison message to block unrelated processing?
8. When would you choose Kafka versus SQS, RabbitMQ, or a synchronous API call?
9. How would you evolve an event schema without breaking existing consumers?
10. How would you use the outbox pattern to keep database state and published events consistent?

9. Distributed Systems & Failure Handling

1. Service A succeeds but service B fails halfway through a business workflow. How would you recover without corrupting state?
2. When should a system prefer strong consistency, and when is eventual consistency acceptable?
3. How would you make an end-to-end workflow safe when both API requests and asynchronous messages can be duplicated?
4. What does 'exactly once' mean in a practical distributed system, and where can it break down?
5. How would you prevent cascading failures when a downstream service becomes slow?
6. How would you design backpressure when incoming work is greater than worker capacity?
7. How would you design graceful load shedding during an overload event?
8. How would you choose timeouts and retry policies across a chain of dependent services?
9. How would you design a job so work is not lost when a worker process crashes or the EC2 instance is terminated?
10. How would you design a reliable scheduled job that can safely recover after missed executions?

10. AWS / Nginx / Networking

1. How would you design a highly available backend using Route 53, ALB, EC2/ECS, Nginx, PostgreSQL, Redis, and S3?
2. How would you decide whether TLS should terminate at the load balancer or Nginx?
3. How would you troubleshoot a request that works by IP address but not through the domain name?
4. How would you preserve and validate client IP information across a proxy/load-balancer chain?
5. How would you decide between ALB, NLB, and API Gateway for a backend service?
6. How would you keep application servers private while exposing the system securely to the internet?
7. How would you design autoscaling for a backend where CPU is not a reliable signal of demand?
8. How would you make an EC2-based deployment highly available across Availability Zones?
9. How would you design zero-downtime deployment on EC2 behind an ALB?
10. One AWS Availability Zone fails during peak traffic. What components must fail over and what should you test beforehand?

11. Security & Multi-Tenancy

1. How would you secure a public FastAPI or NestJS API against common abuse and unauthorized access?
2. How would you design access-token and refresh-token handling for a large application?
3. How would you revoke a refresh token or session without forcing every user to log in again?
4. How would you protect login and password-reset endpoints against brute-force attacks?
5. How would you enforce tenant isolation so one SaaS customer can never access another customer's records?
6. How would you design authorization when users can have both roles and resource-level permissions?
7. How would you securely handle secrets for AWS credentials, database passwords, and third-party API keys?
8. A secret is accidentally committed to source control. What actions should follow?
9. How would you protect a webhook endpoint from spoofed or replayed requests?
10. How would you securely serve private images or documents stored in S3?

12. Delivery, Ownership & Senior Engineering Judgment

1. You are given a major new feature and two junior interns. How would you approach, decompose, delegate, review, and deliver it?
2. Requirements are ambiguous but the delivery date is fixed. How would you reduce risk before implementation?
3. How would you break a large feature into tasks that junior developers can implement safely and independently?
4. What parts of the feature would you own personally and what would you delegate?
5. A junior engineer proposes an architecture you believe will fail under scale. How would you handle the situation?
6. The ideal architecture needs six weeks but the business needs something in two weeks. How would you decide what to ship?
7. A feature is almost complete when a major requirement changes. How would you reassess scope and implementation?
8. Two engineers are building related components and their APIs are drifting apart. How would you prevent integration problems?
9. A production bug is discovered just before release. How would you decide whether to delay the release or proceed?
10. How would you ensure a feature is operable in production, not just functionally correct in development?

Research Basis

- AWS Well-Architected Framework: current AWS guidance organizes architecture evaluation around operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability, with explicit trade-offs between these concerns.
- PostgreSQL current documentation: concurrency control, MVCC, transaction isolation, row-level locking, deadlocks, and serialization failures are core production concerns for backend engineers.
- MongoDB current documentation: production schema design should be query-driven, indexes should be designed for actual query patterns, and sharding requires careful shard-key selection.
- NGINX official documentation: reverse-proxy buffering, upstream connection behavior, timeout settings, proxy caching, and upstream failover are key operational areas for Nginx-backed APIs.
- Node.js official guidance: backend performance depends on not blocking the Event Loop or Worker Pool, making async execution and CPU-bound work important interview topics.
- System-design interview scope was cross-checked against established interview patterns that emphasize requirements, capacity, bottlenecks, caching, load balancing, databases, sharding, and failure handling.

This PDF intentionally excludes answers, hints, and solution walkthroughs. It is designed to be used as a mock-interview checklist.